JDK-> tools+libraries+docs+JRE

Editor & IDE

JRE -> rt.jar + JVM

OOP
- 1. Major Pillars
    - Abstraction
    - Encapsulation
    - Modularity
    - Hirerachy
        - Association
        - Inheritance

- 2. Minor Pillar
    - Typing/Polymorphism
    - Concurrency
    - Persistance

```
add(int n1, int n2){
n1+n2
}

add(int n1, int n2,int n3){
n1+n2+n3
}

add(int n1, int n2,int n3,int n4){
n1+n2+n3+n4
}

main(){
add(10,20);
add(10,20,30);
add(10,20,30,40);
}
```

Function Overloading is an example of CompileTime polymorphism
Function Overriding is an example of RunTime polymorphism

Steps for STS
1. Choose a correct Workspace
    - Generally have daywise workspace
2. Change the perspective to Java
3. Create a new java project
    - give proper name to project
    - select java 1.8 version while creating the project
4. Right Click on src and select new class
    - give proper package name
5. give the classname and select the main method checkbox
6. to run click on run button

```
javac -d ..\..\..\..\bin Program.java
SET CLASSPATH=..\..\..\..\bin
java com.sunbeam.demo.Program
```

Datatype
- It defines 3 things
1. Nature
    - It represents what types of data can be stored
2. Memory
    - How much memory is required to store the data
3. Operations
    - What different types of operations can be carried out on that data

int
double
char

1. Primitive Types (Value Types)
    - Boolean
        - boolean
    - Character
        - char
    - Integral
        - byte,short,int,long
    - Floating-Point
        - float, double

2. Non Primitive types (Reference types)
    - Class
    - Interface
    - Enum
    - Array

Object defines 3 things
- state (fields of the class represents state)
- behaviour(methods of the class represents behaviour)
- identity

```
Point p1;
p1 = new Point();
```

class
- logical entity
- blueprint of an object

object
- physical entity
- instance of a class

```
class Point{
int x;
int y;

void accept(){

}

void display(){

}

void calculateDistance(){

}
}
```